

Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas

INFORME TÉCNICO: IMPLEMENTACIÓN DE ANALIZADORES LÉXICOS EN LEX.

Asignatura: Compiladores y Lenguajes

Autores: Aidan Carrasco, Felipe Merino y Mathew Verdezoto

1. Resumen

El presente informe muestra ejercicios Lex desarrollados para la solución de distintos problemas comunes en el área de backend y teoría de autómatas. El análisis léxico constituye la primera fase de un sistema traductor, el cual transforma un conjunto de flujo continuo de caracteres provenientes de un código fuente en secuencias organizadas con significado semántico llamados tokens.

2. Estructura de un Archivo LEX

Un archivo fuente lex se encuentra estructurado por los siguientes bloques funcionales separados por (%%):

- **Área de declaración de variables:** Se declaran las librerías y variables que se van a utilizar.
- **Área de expresiones regulares:** En esta se adjuntan todas las expresiones regulares necesarias para cada programa y su respectiva impresión en consola si se llega a encontrar algo relacionado con la misma.
- **Área de código:** En esta se empieza a codificar en el lenguaje previamente seleccionado.

3. Implementación y Código Fuente

Desarrollar un analizador léxico que se encargue de reconocer los símbolos básicos y compuestos de la numeración romana, asociando cada token con una acción semántica que asigne un valor entero.

Pruebas y Ejecución:

```
iccd332@DESKTOP-52QQFVV x + v
(base) iccd332@DESKTOP-52QQFVV:~$ mkdir nueve
(base) iccd332@DESKTOP-52QQFVV:~$ cd nueve
(base) iccd332@DESKTOP-52QQFVV:~/nueve$ nano nueve .l
(base) iccd332@DESKTOP-52QQFVV:~/nueve$ nano nueve.l
(base) iccd332@DESKTOP-52QQFVV:~/nueve$ nano entrada.txt
(base) iccd332@DESKTOP-52QQFVV:~/nueve$ lex nueve.l
(base) iccd332@DESKTOP-52QQFVV:~/nueve$ gcc lex.yy.c -lfl
(base) iccd332@DESKTOP-52QQFVV:~/nueve$ ls
a.out  entrada.txt  lex.yy.c  nueve  nueve.l
(base) iccd332@DESKTOP-52QQFVV:~/nueve$ |
```

```
iccd332@DESKTOP-52QQFVV x + v
[1/2] nueve *
%{
    int total=0;
}%
WS      [ \t]+
%%
I        total += 1;
IV       total += 4;
V        total += 5;
IX       total += 9;
X        total += 10;
XL       total += 40;
L        total += 50;
XC       total += 90;
C        total += 100;
CD       total += 400;
D        total += 500;
CM       total += 900;
M        total += 1000;
{WS}    |
\n      return total;
%%
int main ()
{
    int primero, segundo;
    primero = yylex ();
    segundo = yylex ();

    printf ("%d + %d = %d\n", primero, segundo, primero+segundo);
    return 0;
}
```

```
iccd332@DESKTOP-52QQFVV x + v
GNU nano 7.2          entrada.txt *
X
V|
```

```
iccd332@DESKTOP-52QQFVV x + v
(base) iccd332@DESKTOP-52QQFVV:~$ mkdir nueve
(base) iccd332@DESKTOP-52QQFVV:~$ cd nueve
(base) iccd332@DESKTOP-52QQFVV:~/nueve$ nano nueve.l
(base) iccd332@DESKTOP-52QQFVV:~/nueve$ nano nueve.l
(base) iccd332@DESKTOP-52QQFVV:~/nueve$ nano entrada.txt
(base) iccd332@DESKTOP-52QQFVV:~/nueve$ lex nueve.l
(base) iccd332@DESKTOP-52QQFVV:~/nueve$ gcc lex.yy.c -lfl
(base) iccd332@DESKTOP-52QQFVV:~/nueve$ ls
a.out  entrada.txt  lex.yy.c  nueve  nueve.l
(base) iccd332@DESKTOP-52QQFVV:~/nueve$ ./a.out < entrada.txt > salida
.txt
(base) iccd332@DESKTOP-52QQFVV:~/nueve$ cat salida.txt
10 + 15 = 25
(base) iccd332@DESKTOP-52QQFVV:~/nueve$ |
```

```
salida.txt x +
Archivo Editar Ver H1 v B
10 + 15 = 25
```

4. Conclusión

Realizar la implementación de este ejercicio nos permitió entender que un analizador léxico en Lex tiene la capacidad de actuar como un intérprete básico en tiempo real cuando se trata de asociar componentes léxicos con valores semánticos concretos.

